1. Diseño del Esquema de Sincronización

La arquitectura implementada modela el torneo mediante un bloque de memoria compartida POSIX que contiene el estado global del servidor y un arreglo de M estructuras "MesaJuego". Todas las primitivas POSIX se inicializan con el atributo "PTHREAD_PROCESS_SHARED".

- **Emparejamiento y Límite de M Mesas Concurrentes:** Los N clientes ingresan de forma concurrente protegidos por "mutex_servidor" y se sincronizan en pares mediante la variable "cond_emparejamiento". Para que nunca se exceda el límite de M mesas activas simultáneamente, el par formado ejecuta "sem_wait" sobre un semáforo inicializado en M, esperando de forma estrictamente si todas las mesas están ocupadas.
- **Exclusión Mutua del Tablero y Arbitraje del Juez:** Cada mesa activa un hilo **Juez Árbitro** independiente y **13 hilos de fichas** que compiten. El tablero se protege por un esquema de dos niveles:
    1. **Arreglo de Mutexes por Casilla:** Cuando una ficha decide moverse desde una casilla origen hacia destino, adquiere primero los mutex de las casillas involucradas. Esto asegura que si dos o más fichas intentan tomar la misma casilla libre simultáneamente, solo la primera en obtener el cierre de la casilla destino logra proponer la jugada.
    2. **Validación Transaccional con el Juez:** Con las casillas involucradas bloqueadas, la ficha deposita su propuesta en la estructura "PeticionJuez", despierta al hilo Juez y espera en "cond_ficha". El Juez verifica las reglas de adyacencia y captura, actualiza la matriz del tablero, registra el estado visual en "mesa_X.log", evalúa las condiciones de victoria y notifica a la ficha antes de que esta libere los mutex de las casillas.

2. Análisis de Riesgos de Concurrencia

- **Prevención de Interbloqueo (*Deadlock*):** En un tablero donde el Puma se mueve en cualquier dirección y los Perros avanzan entre casillas adyacentes, si la ficha A intenta moverse de la casilla "u" a la "v" bloqueando primero "u" y luego "v", mientras otra ficha B intenta operar sobre "v" y "u" en orden inverso, se produciría una espera circular (*deadlock*). Para eliminar por completo la condición de espera circular, se implementó la función "bloquear_casillas_ordenadas()", la cual ordena ascendentemente los identificadores numéricos de todas las casillas involucradas antes de invocar "pthread_mutex_lock". Ademas, existe una jerarquía estricta de bloqueos siempre se adquieren primero los "mutex_casilla" y posteriormente "mutex_mesa".
- **Prevención de Bloqueo Activo (*Livelock*):** Si dos fichas compiten reiteradamente por la misma intersección o si el canal de validación del Juez está ocupado por otra jugada en curso, una ficha que encuentra "peticion.pendiente == true" libera inmediatamente sus "mutex_casilla" y vuelve a su ciclo de decisión con un retardo aleatorio entre "DELAY_MIN_MS" y "DELAY_MAX_MS" (50 a 300 ms) generado con "rand_r" y semillas independientes por hilo, rompiendo cualquier sincronía cíclica entre hilos.
- **Mitigación de Inanición y Resiliencia:** Todas las esperas se realizan sobre primitivas bloqueantes del kernel y retardos con nanosleep, evitando esperas activas

que acaparen la CPU. Además, el Juez utiliza “pthread_cond_timedwait” cada 100 ms para auditar los “heartbeats” de ambos jugadores; si un jugador aborta o falla internamente, el Juez detecta la anomalía, aborta la partida declarando ganador al oponente por abandono y libera las primitivas y la mesa. Ante una señal “SIGINT” (Ctrl+C), el manejador activa una bandera “sig_atomic_t”, detiene nuevas llegadas, despierta a todas las variables de condición y semáforos, ejecuta “pthread_join” de todos los hilos activos, destruye las primitivas y desvincula la memoria compartida sin fugas de memoria.

3. Evaluación del Diseño

- **Rendimiento (*Throughput* y Latencia):** Al separar los cierres por casilla del arbitraje del Juez, la contención directa entre piezas que se encuentran en zonas distintas del tablero se reduce durante la fase de adquisición de casillas, manteniendo la latencia de validación en el orden de microsegundos frente a los ciclos de decisión de 50 a 300 ms.
- **Complejidad vs. Concurrencia:** Un único mutex global que bloqueara todo el ciclo de pensamiento y movimiento serializaría artificialmente el juego, mientras que una configuración completamente libre de Juez impediría cumplir de forma atómica con el requisito de validación arbitral y registro coherente del estado del tablero. La combinación de bloqueo ordenado por casilla con validación síncrona del Juez garantiza consistencia en los registros, ausencia matemática de condiciones de carrera y cumplimiento estricto de las reglas del Komikan en tiempo real.